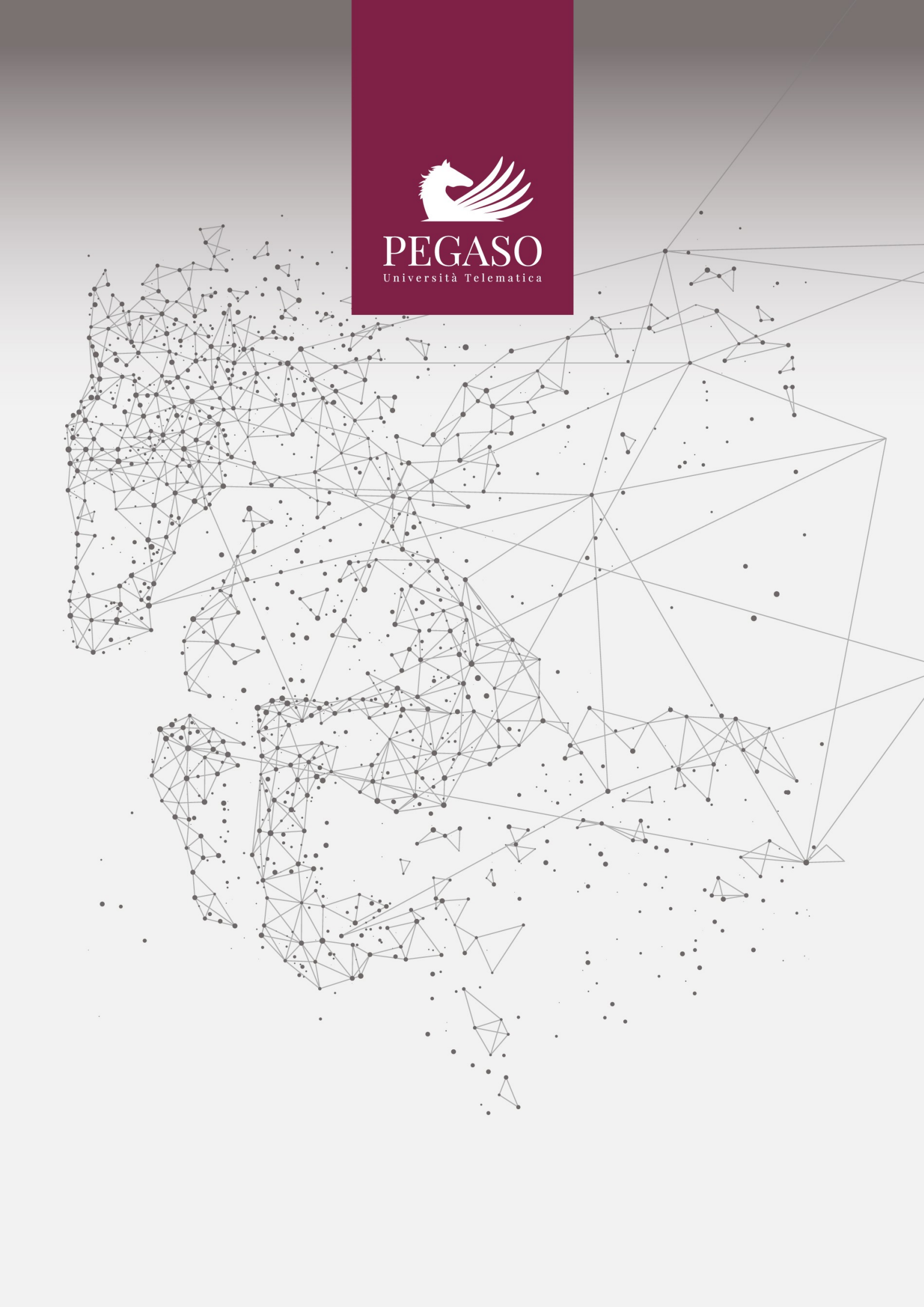

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE....3
2. DISTRIBUTED TRANSACTION PROCESSING....4
3. ENTERPRISE APPLICATION INTEGRATION 7
BIBLIOGRAFIA 9

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

Un'altra categoria significativa di sistemi distribuiti può essere riscontrata in quelle aziende che devono gestire una pletora di applicazioni in rete, ma hanno verificato che l'interoperabilità è una sfida difficile da superare.

In passato, lavorare con un'infrastruttura che semplificava l'integrazione delle applicazioni in un sistema informativo a livello aziendale ha portato allo sviluppo di molte delle soluzioni middleware attualmente in uso.

L'integrazione può avvenire su diversi livelli, che possono essere distinti in modo chiaro. La maggior parte delle volte, un'applicazione in rete non è altro che un server che gestisce l'applicazione in questione (e spesso include un database) e la rende accessibile ai programmi che si trovano altrove, noti come client. Questi client avviano la comunicazione con il server inviando una richiesta per eseguire una particolare operazione. Dopo il completamento dell'azione richiesta, il client riceve una risposta.

L'integrazione al livello base consente ai client di combinare più richieste, potenzialmente anche per server diversi, in un'unica richiesta più grande e di eseguire tale richiesta come transazione distribuita. La cosa più importante da ricordare è che tutte le richieste saranno eseguite o nessuna di esse lo farà. Risulta ovvio che l'integrazione dovrebbe avvenire anche consentendo alle applicazioni di comunicare direttamente tra loro. Ciò si è reso necessario nel tempo, man mano che le applicazioni diventavano più complesse e venivano gradualmente suddivise in singoli componenti (in particolare differenziando i componenti di database dai componenti di elaborazione), il che ha dimostrato che anche l'integrazione delle applicazioni sarebbe stata necessaria. Di conseguenza, è emerso un importante framework industriale che si focalizza sull'integrazione delle applicazioni aziendali, Enterprise Application Integration (EAI).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Distributed transaction processing

Per rendere la definizione dei concetti di questa lezione il più chiara possibile, l'attenzione sarà rivolta per la maggior parte sulle applicazioni di database. Le transazioni sono il metodo standard per eseguire operazioni su un database, quando lo si sta effettivamente utilizzando. La programmazione che utilizza transazioni richiede l'utilizzo di primitive specializzate, che devono essere fornite dal sistema distribuito sottostante o da un software runtime system. La Tabella 1 presenta una serie di primitive di transazione a titolo di esempio.

Primitiva	Descrizione
BEGIN_TRANSACTION	Contrassegnare l'inizio di una transazione
END_TRANSACTION	Terminare la transazione e provare a eseguire il commit
ABORT_TRANSACTION	Terminare la transazione e ripristinare i vecchi valori
READ	Leggere dati da un file, una tabella o altro
WRITE	Scrivere dati in un file, una tabella o in altro modo

Tabella 1

L'elenco preciso delle primitive è determinato dai tipi di oggetti utilizzati nella transazione. Le funzioni primitive per l'invio, la ricezione e l'inoltro della posta elettronica potrebbero essere incluse in un sistema di posta elettronica. Potrebbero essere molto differenti nel contesto di un sistema contabile. Al contrario, READ e WRITE sono esempi molto comuni, usate nella maggior parte dei contesti.

All'interno di una transazione, è anche consentito includere dichiarazioni regolari, chiamate a procedure e simili. In particolare, le chiamate di procedura remota (RPC), note anche come chiamate di procedura a server remoti, sono spesso incapsulate all'interno di una transazione, portando a ciò che è noto come RPC transazionale. Per delimitare una transazione, vengono utilizzate le parole chiave BEGIN TRANSACTION e END TRANSACTION. Il nucleo della transazione è costituito dalle altre operazioni. Una transazione è definita dal fatto che tutte queste operazioni vengono eseguite o nessuna di esse lo è. A

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

seconda dell'implementazione, potrebbero essere operazioni come chiamate di sistema, procedure di libreria o altro.

Le transazioni sono caratterizzate da quattro proprietà distinte, una delle quali è una caratteristica "tutto o niente". Per essere più specifici, le transazioni sono conformi alle cosiddette proprietà ACID, che sono le seguenti:

- Atomica: la transazione appare al mondo esterno come se fosse un singolo evento.
- Coerente: la transazione non viola in alcun modo alcuna invariante di sistema
- Isolata: le transazioni che si svolgono contemporaneamente non interferiscono l'una con l'altra
- Durevole: le modifiche apportate da una transazione sono irreversibili una volta che la transazione è stata portata a termine

Nei sistemi distribuiti, le transazioni sono spesso costruite da una serie di sottotransazioni, che si uniscono per formare una transazione come rappresentato in Figura 1.

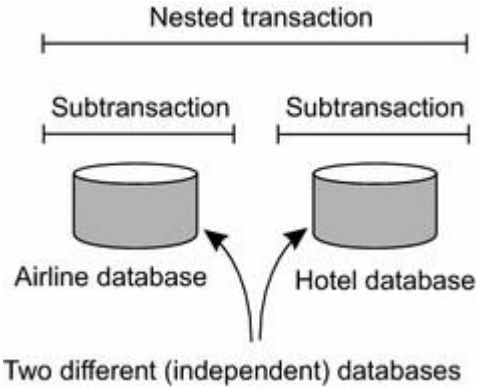


Figura 1 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2016). A brief introduction to distributed systems. Computing

Al fine di migliorare le prestazioni o semplificare la programmazione, una transazione di primo livello si può separare in due transazioni figlie che vengono eseguite in parallelo tra loro, su computer diversi. Ognuna di queste transazioni è anche in grado di effettuare una o più sottotransazioni, oltre a poter generare altre transazioni figlie.

Le sottotransazioni danno luogo a un problema che spesso passa inosservato, il quale tuttavia ha ripercussioni significative. Si supponga che una transazione generi diverse sottotransazioni in parallelo e che una di queste sottotransazioni venga quindi sottoposta a commit, quindi venga portata a termine, di fatto rendendo visibili i risultati alla transazione padre. Dopo un'ulteriore elaborazione, la transazione padre esegue il rollback, riportando l'intero sistema nello stato in cui si trovava prima dell'avvio della transazione di primo livello. Come risultato diretto di ciò, è necessario invertire i risultati della

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

sottotransazione di cui è stato eseguito il commit. Pertanto, la permanenza discussa in precedenza si applica solo alle transazioni al livello più alto.

A causa del fatto che le transazioni possono essere nidificate ad una profondità arbitraria, è necessario un notevole sforzo in fase di amministrazione per garantire che tutto venga gestito correttamente. La semantica, d'altra parte, è definita molto chiaramente. Quando inizia una transazione o una sottotransazione, viene creata una copia privata di tutti i dati dell'intero sistema, per essere modificata in qualsiasi modo ritenga opportuno. Se la transazione viene interrotta, viene semplicemente eliminata ed è come se non ci fosse mai stata. Nel caso in cui sia portata a termine, il suo spazio privato prenderà il posto di quello governato dalla transazione genitore. Pertanto, se viene eseguito il commit di una sottotransazione e successivamente viene avviata un'altra transazione secondaria, i risultati prodotti dalla prima transazione secondaria sono visibili alla seconda transazione secondaria. Allo stesso modo, nel caso in cui una transazione di chiusura (livello superiore) fallisca, anche ciascuna delle sue sottotransazioni sottostanti deve fallire per mantenere la coerenza. Inoltre, se vengono avviate più transazioni contemporaneamente, il risultato è lo stesso come se fossero state elaborate una dopo l'altra in un ordine arbitrario.

Nei sistemi distribuiti, le transazioni nidificate svolgono un ruolo importante perché offrono un modo naturale di distribuire una transazione su più macchine. Di conseguenza, queste transazioni sono essenziali. Sono organizzate in modo logicamente coerente con la transazione iniziale. Ad esempio, una transazione che comporta la pianificazione di un viaggio e deve includere la prenotazione di tre voli separati, può logicamente essere suddivisa in tre sottotransazioni separate. Ognuna di queste sottotransazioni può essere gestita singolarmente e indipendentemente dalle altre due.

Agli albori dei sistemi middleware aziendali, il componente che gestiva le transazioni distribuite (o nidificate) costituiva il nucleo per l'integrazione delle applicazioni a livello di server o database. Questo componente è stato indicato come un monitor di elaborazione delle transazioni, o TP monitor. L'obiettivo principale era fornire ad un'applicazione un modello di programmazione transazionale in modo che l'applicazione potesse accedere a più server e database contemporaneamente. In sostanza, TP monitor era responsabile del coordinamento delle sottotransazioni mediante un protocollo standard noto come commit distribuito. Una cosa importante da notare è che le applicazioni che volevano coordinare più sottotransazioni in una singola transazione non dovevano implementare questo coordinamento da sole per farlo, proprio perché il monitor TP gestiva tutto automaticamente. Questo è esattamente il punto in cui entra in gioco il middleware, poiché implementa servizi utili per molte applicazioni, evitando che gli sviluppatori di applicazioni debbano reimplementare ripetutamente tali servizi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Enterprise application integration

Come è stato detto, più le applicazioni venivano disaccoppiate dai database su cui erano costruite, più diventava ovvio che le strutture dovevano integrare le applicazioni indipendentemente dai loro database. In particolare, i componenti dell'applicazione dovrebbero essere in grado di comunicare direttamente tra loro piuttosto che basarsi esclusivamente sul modello di richiesta e risposta tradizionalmente supportato dai sistemi di elaborazione delle transazioni. Il requisito della comunicazione tra applicazioni ha portato allo sviluppo di un'ampia varietà di modelli di comunicazione. Il concetto principale alla base di questi modelli era che le applicazioni già in uso potevano condividere direttamente le informazioni tra loro, come illustrato nella Figura 2.

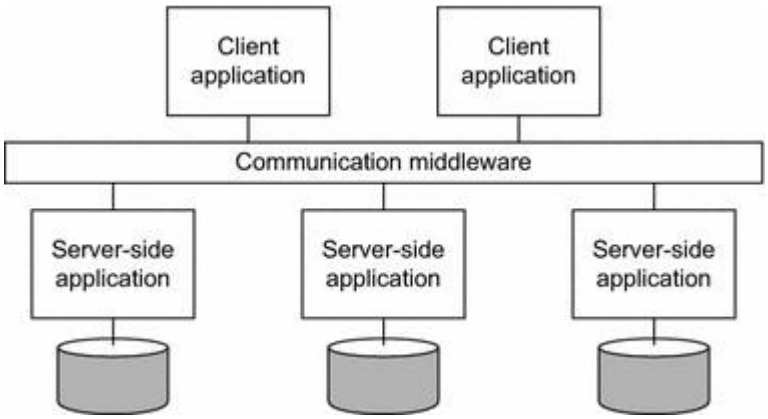


Figura 2 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2016). A brief introduction to distributed systems. Computing

Esistono alcune categorie distinte di middleware di comunicazione tra cui scegliere. Utilizzando le chiamate di procedura remota (RPC), un componente dell'applicazione può effettivamente inviare una richiesta a un altro componente dell'applicazione eseguendo una chiamata di procedura locale, in modo che quindi la richiesta viene inserita nel pacchetto come messaggio e inviata al destinatario della chiamata. A seguito della RPC, anche il risultato verrà rispedito indietro e restituito all'applicazione. A causa della crescente domanda di tecnologia degli oggetti, sono stati sviluppati vari metodi che consentono di effettuare chiamate a oggetti remoti, dando origine al concetto che ora è noto come "invocazioni di metodi remoti" o, in inglese, remote methods invocations (RMI). La differenza principale tra un RMI e un RPC è che il primo funziona con gli oggetti, mentre il secondo funziona con le funzioni. Sia RPC che RMI presentano lo svantaggio che il chiamante e il destinatario della chiamata devono essere operativi al momento della comunicazione. Questa è una limitazione di entrambi i protocolli. Oltre a questo, è essenziale che abbiano

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

familiarità con la terminologia corretta da utilizzare quando comunicano l'uno all'altro. Questo stretto accoppiamento è spesso percepito come uno svantaggio significativo, che è ciò che ha portato allo sviluppo di qualcosa che ora è noto come middleware orientato ai messaggi, o message-oriented middleware (MOM). In questo scenario, le applicazioni invieranno messaggi ai punti di contatto logici. Allo stesso modo, le applicazioni possono segnalare il loro interesse a ricevere un particolare tipo di messaggio, a quel punto il middleware di comunicazione garantirà che i messaggi in questione vengano inviati alle applicazioni appropriate. Questi cosiddetti sistemi di pubblicazione/sottoscrizione costituiscono un'importante sottocategoria di sistemi distribuiti, che sta vivendo una rapida espansione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.
- M. van Steen and A.S. Tanenbaum, A brief introduction to distributed systems. Computing, 2016.
- Hohpe G. and Woolf B. Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions. Addison-Wesley, Reading, MA., 2004.
- Gray J. and Reuter A. Transaction Processing: Concepts and Techniques. Morgan Kaufman, San Mateo, CA., 1993.
- Bernstein P. and Newcomer E. Principles of Transaction Processing. Morgan Kaufman, San Mateo, CA., 2nd edition, 2009.
- Bernstein P. Middleware: A Model for Distributed System Services. Communications of the ACM, 39(2):87–98, Feb. 1996.
- Alonso G., Casati F., Kuno H., and Machiraju V. Web Services: Concepts, Architectures and Applications. Springer-Verlag, Berlin, 2004.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).